

Banking Data ETL Pipeline on AWS

Secure, scalable data engineering for financial transactions

S3

Glue

Step Functions

Athena

Redshift

QuickSight

AUTHOR

AXIS

GITHUB

<https://github.com/naimuddin801?tab=repositories>

LINKEDIN

www.linkedin.com/in/nu0396975

Portfolio Project | 2026

1

Project Overview

This project builds an end-to-end, serverless ETL pipeline on AWS that ingests banking data (customer accounts, card and account transactions, loan records), cleans and validates it, masks sensitive fields, and publishes analytics-ready datasets for reporting, risk and fraud analysis. It is designed around the requirements that matter in banking: **data security, auditability, data quality and reliability**.

Problem statement

Banks hold data across core systems, card processors and flat-file feeds. Analysts struggle with inconsistent formats, duplicate records, unmasked PII and slow manual reporting. A governed, automated pipeline solves this by creating a single trusted source for analytics.

Goals

- Automate daily batch ingestion from multiple source systems into a central data lake
- Enforce data quality checks and reject or quarantine bad records
- Mask / tokenize PII (account numbers, names, phone, email) before data reaches analysts
- Produce partitioned Parquet datasets and a star-schema warehouse for fast queries
- Provide monitoring, alerting, and a full audit trail using managed AWS services
- Deploy everything as Infrastructure as Code so it is repeatable

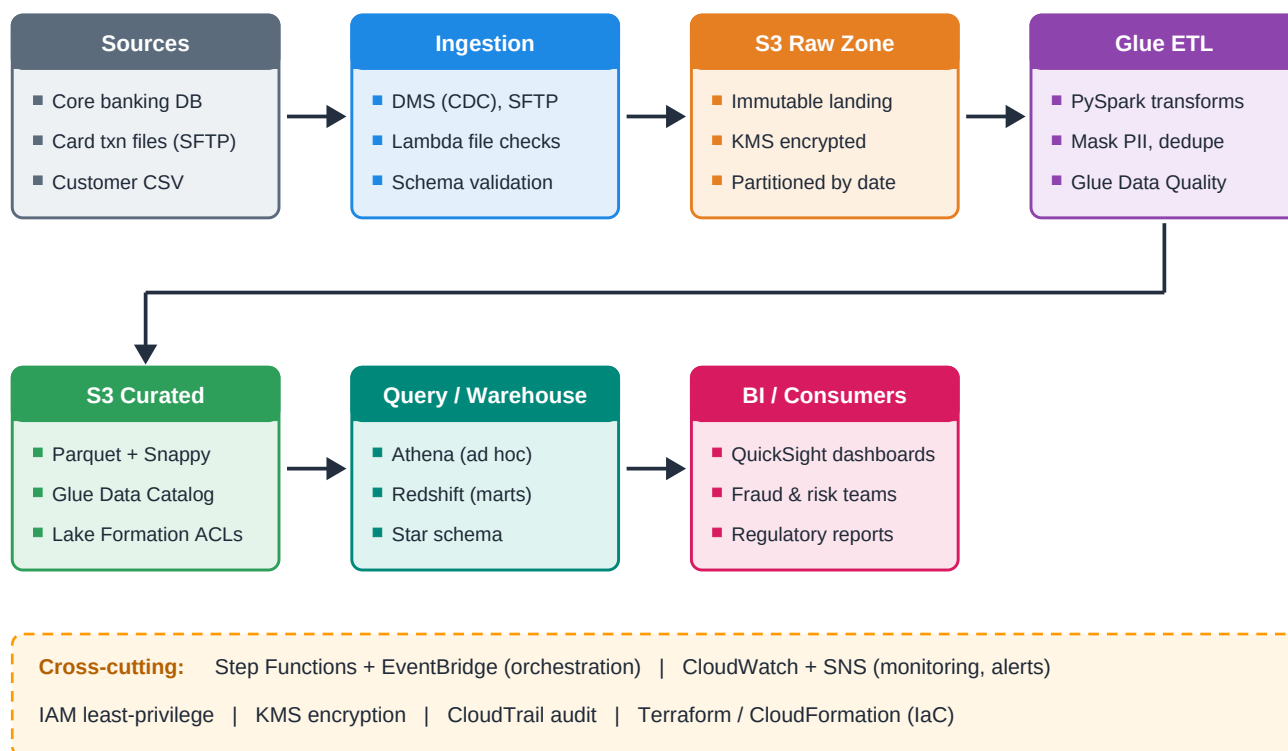
Tech stack

Layer	Services / Tools
Storage	Amazon S3 (raw, curated, quarantine zones), Parquet + Snappy
Ingestion	AWS DMS (CDC), AWS Transfer Family (SFTP), AWS Lambda
Transformation	AWS Glue (PySpark), Glue Data Quality, Glue Data Catalog
Orchestration	AWS Step Functions, Amazon EventBridge
Analytics	Amazon Athena, Amazon Redshift, Amazon QuickSight
Security	IAM, KMS, Lake Formation, Secrets Manager, CloudTrail
Monitoring	CloudWatch (logs, metrics, alarms), SNS notifications
DevOps	Terraform or CloudFormation, GitHub Actions (CI/CD), Python 3.11

2

Architecture

AWS Architecture: Banking ETL Pipeline



Data flow

1	Ingest: DMS streams changes from the core banking database; card transaction files arrive via SFTP. Both land untouched in the S3 raw zone, partitioned by source/year/month/day.
2	Validate: An S3 event triggers a Lambda that checks file name, schema, and checksum. Invalid files move to a quarantine prefix and raise an SNS alert.
3	Transform: A Glue PySpark job standardizes types and dates, removes duplicates, applies business rules, and masks PII with hashing or tokenization.
4	Quality gate: Glue Data Quality rules (completeness, uniqueness, valid ranges) run before publishing. A failed ruleset stops the workflow.
5	Publish: Clean data is written to the curated zone as partitioned Parquet and registered in the Glue Data Catalog.
6	Serve: Athena supports ad hoc SQL; Redshift holds fact and dimension tables; QuickSight dashboards sit on top.
7	Orchestrate: Step Functions coordinates the stages with retries and error handling, scheduled daily by EventBridge.

Key design decisions

Decision	Rationale
Serverless (Glue, Lambda, Step Functions)	No cluster management; pay per use; scales with data volume
Three S3 zones (raw / curated / quarantine)	Keeps an immutable source of truth and isolates bad data for review
Parquet with date partitioning	Columnar format cuts Athena scan cost and improves query speed
Mask PII in the transform stage	Analysts never see raw sensitive fields; supports least-privilege access
Idempotent jobs with job bookmarks	Safe re-runs without duplicating records

3

Data Model

Source entities are modeled into a star schema in Redshift for reporting.

Table	Type	Key columns
fact_transactions	Fact	txn_id, account_key, date_key, merchant_key, amount, currency, txn_type, status
dim_customer	Dimension	customer_key, customer_token (masked), segment, city, onboarding_date
dim_account	Dimension	account_key, account_token (masked), product_type, open_date, status
dim_date	Dimension	date_key, date, month, quarter, year, is_weekend
dim_merchant	Dimension	merchant_key, merchant_name, category_code, country

Data quality rules

- **Completeness:** txn_id, account_id, amount and txn_timestamp must not be null
- **Uniqueness:** txn_id unique per day after deduplication
- **Validity:** amount is numeric; currency is in an approved ISO-4217 list; timestamps are not in the future
- **Referential integrity:** every transaction maps to an existing account
- **Handling:** failing rows go to the quarantine zone with a reason code; a run fails if the error rate crosses a set threshold

Sample transformation (Glue / PySpark)

```
from pyspark.sql import functions as F
df = spark.read.parquet("s3://bank-etl-raw/transactions/")
clean = (df
    .dropDuplicates(["txn_id"])
    .withColumn("amount", F.col("amount").cast("decimal(18,2)"))
    .withColumn("txn_ts", F.to_timestamp("txn_timestamp"))
    .filter(F.col("amount").isNotNull() & F.col("account_id").isNotNull())
    # PII masking: salted SHA-256 token instead of the raw account number
    .withColumn("account_token",
        F.sha2(F.concat(F.col("account_id"), F.lit(SALT)), 256))
    .drop("account_id", "customer_name", "phone", "email")
    .withColumn("dt", F.to_date("txn_ts")))
(clean.write.mode("overwrite")
    .partitionBy("dt")
    .parquet("s3://bank-etl-curated/transactions/"))
```

4 Security and Governance

Because the data is financial and personal, security is built into every layer rather than added afterwards.

Area	Implementation
Encryption at rest	S3, Redshift and Glue job outputs encrypted with customer-managed KMS keys
Encryption in transit	TLS enforced; S3 bucket policy denies non-HTTPS requests
Access control	IAM roles per service with least privilege; Lake Formation column-level permissions on curated data
PII protection	Masking/tokenization in Glue; raw zone restricted to the pipeline role only
Secrets	Database credentials stored in Secrets Manager, never in code
Network	VPC endpoints for S3 and Glue; private subnets for DMS and Redshift
Audit	CloudTrail and S3 access logs retained for audit; data lineage via the Glue Data Catalog
Retention	S3 lifecycle rules move older raw data to Glacier per retention policy

Monitoring and reliability

- CloudWatch alarms on job failures, run duration and data-quality failures, delivered through SNS (email or Slack)
- Step Functions retries with exponential backoff; failed runs land in a dead-letter path for investigation
- Idempotent, re-runnable jobs so a failed day can be reprocessed without side effects
- Dashboards tracking rows processed, rejected rows, and pipeline latency

Cost optimization

- Parquet + partitioning reduces Athena data scanned, which lowers query cost
- Glue auto scaling and right-sized workers; jobs run on a schedule rather than continuously
- S3 lifecycle tiering and Redshift pause/resize for non-production environments

5 Deployment and CI/CD

- All infrastructure (buckets, roles, Glue jobs, Step Functions, alarms) defined in Terraform or CloudFormation
- GitHub Actions pipeline: lint (flake8), unit tests (pytest with a local Spark session), terraform validate/plan, then deploy to dev and prod
- Separate dev and prod AWS accounts or environments; synthetic data used in non-production

Suggested repository structure

```

banking-etl-aws/
├── infra/           # Terraform / CloudFormation
├── glue_jobs/      # PySpark ETL scripts
├── lambda/         # file validation functions
├── statemachine/   # Step Functions definition (ASL)
├── sql/           # Redshift DDL and mart queries
├── tests/         # pytest unit + data quality tests
├── data/sample/    # synthetic banking data generator
├── .github/workflows/ # CI/CD
└── README.md

```

6 Results and Metrics

Fill in these figures from your own test runs before sharing. Use real measured numbers; recruiters often ask how you measured them.

Metric	Your result
Records processed per daily run	[e.g. number of rows]
End-to-end pipeline runtime	[minutes]
Rows quarantined by quality rules	[% of total]
Athena scan reduction (CSV vs Parquet)	[% or GB]
Estimated monthly AWS cost	[USD]

7 Challenges and Learnings

- **Schema drift:** source files changed columns; handled with explicit schemas and schema-validation in Lambda
- **Duplicate and late-arriving data:** solved with deterministic keys, deduplication and partition overwrite
- **PII handling:** balancing analyst usability with privacy led to salted tokens that still allow joins
- **Small-file problem:** compacted output files in Glue to improve Athena performance

8

Future Enhancements

- Near-real-time fraud signals using Kinesis Data Streams and Lambda
- Data lineage and cataloging with DataZone or OpenLineage
- Anomaly detection on transaction patterns with SageMaker
- dbt on Redshift for tested, version-controlled SQL models

Skills demonstrated

AWS data engineering, PySpark, data modeling (star schema), data quality, security and compliance design, orchestration, Infrastructure as Code, CI/CD, cost optimization.